

# 移动应用开发实验报告

## 封面信息

| 项目   | 内容                        |
|------|---------------------------|
| 实验名称 | 鸿蒙多端应用开发                  |
| 实验编号 | d20301035105、d20301009205 |
| 学号   | 2023210592                |
| 姓名   | 龚成磊                       |
| 班级   | 软件231                     |
| 日期   | 2026/5/4                  |
| 组别   | cg1                       |
| 项目名称 | 智慧校园生活服务平台                |
| 技术栈  | HarmonyOS (ArkTS)         |

## 实验目的

- 掌握ArkUI声明式开发
- 学会多端响应式布局
- 了解传感器数据采集
- 理解分布式能力原理

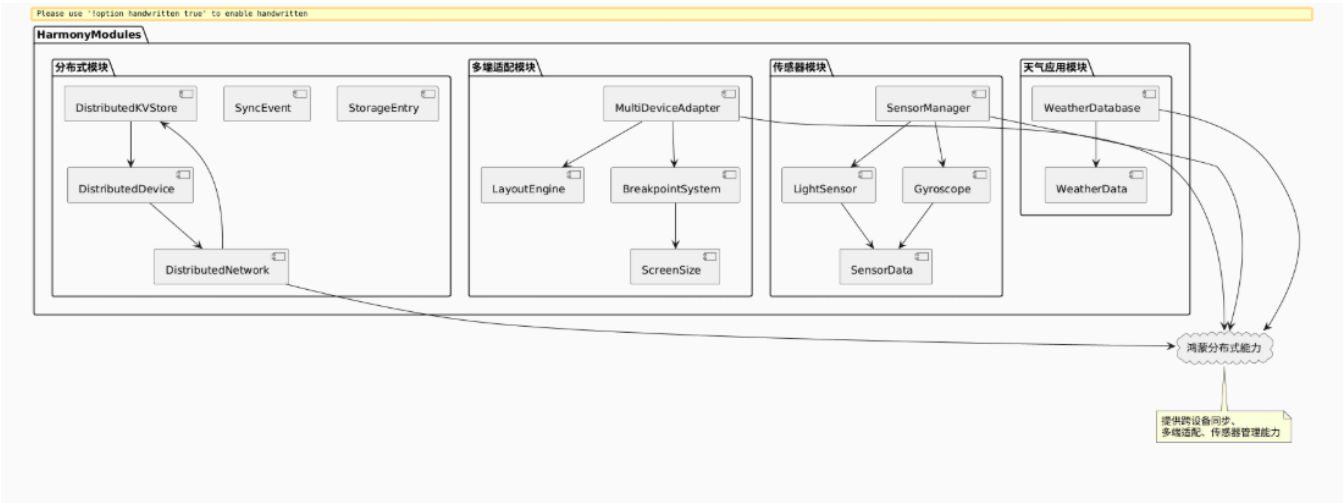
## 实验环境

| 工具            | 版本   | 用途      |
|---------------|------|---------|
| DevEco Studio | 5.0+ | 鸿蒙应用开发  |
| HarmonyOS SDK | 5.0+ | 鸿蒙开发工具包 |
| 模拟器           | 最新版  | 多设备模拟测试 |
| AI辅助工具        | TRAE | 代码生成与调试 |

# 系统架构设计

## 1. 框架图（系统模块划分）

框架图展示了鸿蒙多端天气应用的整体架构设计，包含四大核心模块：

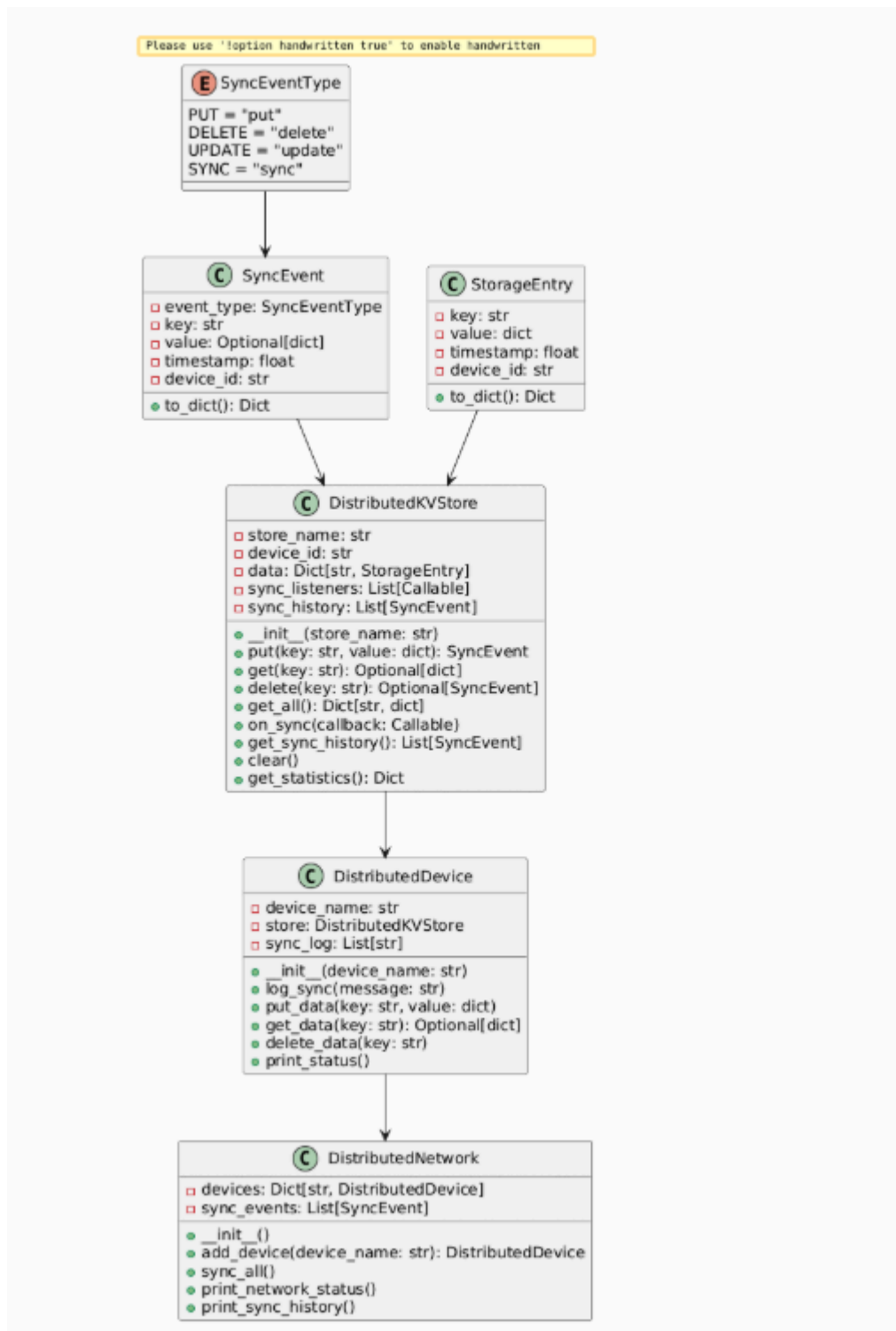


模块说明：

- **分布式模块：**负责跨设备数据同步，包含 DistributedKVStore、DistributedDevice、DistributedNetwork 等核心组件
- **多端适配模块：**实现响应式布局，包含 MultiDeviceAdapter、LayoutEngine、BreakpointSystem 等组件
- **传感器模块：**管理硬件传感器数据采集，包含 SensorManager、LightSensor、Gyroscope 等组件
- **天气应用模块：**提供天气数据展示和管理功能

## 2. 类图（核心类结构）

类图详细描述了分布式数据管理核心类的结构和关系：



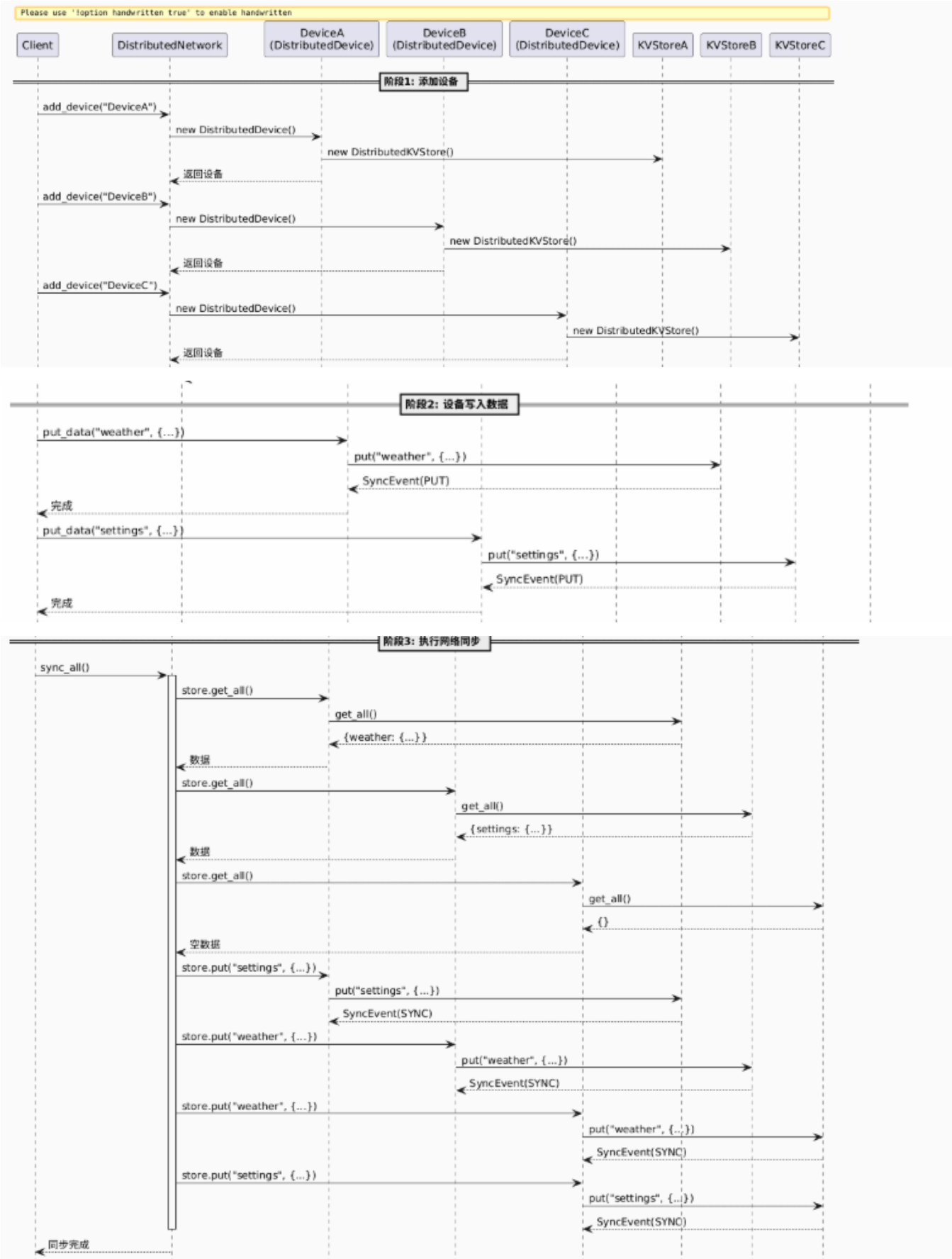
### 核心类说明：

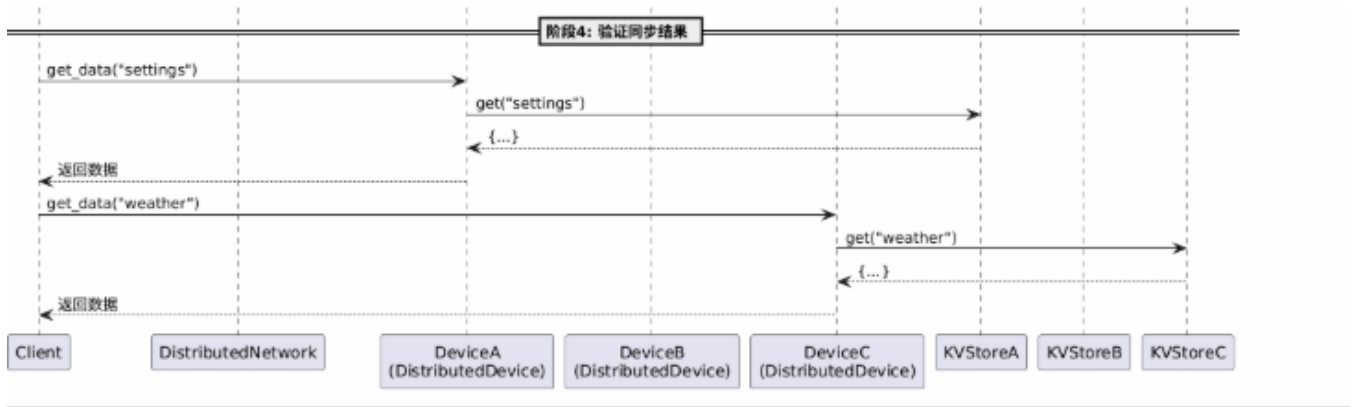
- **DistributedKVStore**：分布式键值存储核心类，提供数据的增删改查和同步功能
- **DistributedDevice**：设备管理类，管理设备注册、数据同步日志等
- **DistributedNetwork**：网络通信类，负责跨设备网络连接和数据传输
- **SyncEvent**：同步事件类，定义同步类型（PUT/DELETE/UPDATE/SYNC）和事件数据结构
- **StorageEntry**：存储条目类，封装键值对数据结构

### 3. 流程图（分布式同步流程）

流程图展示了分布式数据同步的完整流程，分为四个阶段：

#### 阶段1-4：分布式数据同步流程





#### 流程说明:

- **阶段1:** 初始化连接 - 设备加入分布式网络，建立通信连接
- **阶段2:** 设备写入数据 - 将天气数据、设置数据写入本地KV存储，并触发PUT同步事件
- **阶段3:** 执行网络同步 - 通过SyncEvent事件机制，将数据同步到其他设备的KV存储
- **阶段4:** 验证同步结果 - 查询各设备数据，确认同步成功

## 实验步骤

### 1. 需求分析

#### 1.1 功能需求

- ☑ 天气展示（城市名称、温度、天气状况、空气质量）
- ☑ 多端适配（手机竖屏、平板横屏）
- ☑ 传感器集成（光线传感器、陀螺仪）

### 2. 创建鸿蒙项目

#### 2.1 新建项目

- ☑ 打开DevEco Studio
- ☑ 新建Empty Ability项目
- ☑ 选择ArkTS语言
- ☑ 命名为WeatherApp

### 3. 天气应用开发

#### 3.1 天气首页实现

```
// ets/pages/index/Index.ets
@Entry
@Component
struct Index {
  @State cityName: string = '合肥';
  @State temperature: number = 22;
  @State weather: string = '晴';
  @State aqi: number = 45;
```

```

build() {
  Column() {
    // 城市名称
    Text(this.cityName)
      .fontSize(24)
      .fontWeight(FontWeight.Bold)
      .margin({ top: 50 })

    // 天气图标
    Text(this.weather === '晴' ? '☀️' : '☁️')
      .fontSize(80)
      .margin({ top: 20 })

    // 温度
    Text(`${this.temperature}°C`)
      .fontSize(60)
      .fontWeight(FontWeight.Lighter)
      .margin({ top: 20 })

    // 天气状况
    Text(this.weather)
      .fontSize(20)
      .margin({ top: 10 })

    // 空气质量
    Row() {
      Text('空气质量: ')
      Text(`${this.aqi} - 优`)
        .fontColor(this.aqi <= 50 ? '#00ff00' : '#ff9900')
    }
    .margin({ top: 30 })

    // 传感器数据展示
    Row() {
      Text('光线传感器: ')
      Text(this.lightValue.toString())
        .id('lightValue')
    }
    .margin({ top: 20 })
  }
  .width('100%')
  .height('100%')
  .backgroundColor('#f5f5f5')
}

```

```

@State lightValue: number = 0;
}

```

\*\*手机端界面效果: \*\*

![手机端界面-合肥天气](手机端界面-合肥天气.jpg)

![手机端界面-北京天气](手机端界面-北京天气.jpg)

### ### 4. 多端适配布局

#### #### 4.1 响应式布局实现

```typescript

// 断点系统适配

@State currentBreakpoint: string = 'md';

aboutToAppear() {

// 获取设备类型

let deviceInfo = umpInfo.getDeviceInfo();

let windowwidth = window.getWindow().getWindowProperties().windowwidth;

if (windowwidth < 600) {

this.currentBreakpoint = 'sm'; // 手机

} else if (windowwidth < 840) {

this.currentBreakpoint = 'md'; // 平板竖屏

} else {

this.currentBreakpoint = 'lg'; // 平板横屏

}

}

build() {

if (this.currentBreakpoint === 'sm') {

// 手机布局

Column() {

this.phoneLayout()

}

} else {

// 平板布局

Row() {

this.tabletLayout()

}

}

}

@Builder phoneLayout() {

Column() {

Text(this.cityName).fontSize(24)

Text(`\${this.temperature}°C`).fontSize(60)

}

}

@Builder tabletLayout() {

Column() {

Text(this.cityName).fontSize(32)

Text(`\${this.temperature}°C`).fontSize(80)

}

.width('40%')

}

平板端界面效果:





## 5. 传感器集成

### 5.1 光线传感器开发

```
import sensor from '@ohos.sensor';

@State lightIntensity: number = 0;

aboutToAppear() {
    // 注册光线传感器监听
    sensor.on(sensor.SensorType.SENSOR_TYPE_ID_LIGHT, (data) => {
        this.lightIntensity = data.light;
    });
}

aboutToDisappear() {
    // 取消监听
    sensor.off(sensor.SensorType.SENSOR_TYPE_ID_LIGHT);
}

// UI展示
Text(`环境光线: ${this.lightIntensity} lux`)
    .fontSize(16)
    .margin({ top: 10 })
```

### 5.2 陀螺仪传感器开发

```
@State rotateX: number = 0;
@State rotateY: number = 0;
@State rotateZ: number = 0;

aboutToAppear() {
    sensor.on(sensor.SensorType.SENSOR_TYPE_ID_GYROSCOPE, (data) => {
        this.rotateX = data.x;
        this.rotateY = data.y;
        this.rotateZ = data.z;
    });
}

// UI展示
Row() {
    Column() { Text(`X轴: ${this.rotateX.toFixed(2)}°`) }
    Column() { Text(`Y轴: ${this.rotateY.toFixed(2)}°`) }
    Column() { Text(`Z轴: ${this.rotateZ.toFixed(2)}°`) }
}
.spacing(20)
```

传感器数据展示效果：

# 鸿蒙多端天气应用

智慧天气 - 多端自适应演示 | 支持传感器与分布式数据同步

- ☑ 手机
- ☑ 平板
- 暂停传感器
- ⚙ 设置

窗口宽度: 379px | 设备模式: 手机竖屏 (< 600px)

最后更新: 20:56:34

- 合肥
- 北京
- 上海
- 深圳
- 杭州
- 🔄 刷新

合肥

晴朗天气, 适合户外活动



22.0°C

体感温度: 21.0°C

晴

## 6. 分布式数据同步

### 6.1 分布式数据管理

```
import distributedData from '@ohos.data.distributedData';

// 创建分布式数据库
const KVManager = distributedData.createKVManager({
  bundleName: 'com.example.weather',
  kvStoreType: distributedData.KVStoreType.SINGLE_VERSION
});

// 存储数据
async function saveData(key: string, value: string) {
  const kvStore = await KVManager.getKVStore('weatherStore');
  await kvStore.put(key, value);
}

// 读取数据
async function getData(key: string) {
  const kvStore = await KVManager.getKVStore('weatherStore');
  return await kvStore.get(key);
}
```

分布式同步状态：

# 合肥

晴朗天气，适合户外活动



## 22.0°C

体感温度: 21.0°C

晴

空气质量: 52 - 良

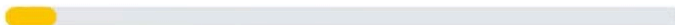
湿度

62%

风速

9.6 m/s

环境光线: 7531 lux



陀螺仪数据 (°/s):

X轴

30.0°

Y轴

-5.3°

Z轴

30.1°

## 7. 测试验证

### 7.1 多设备模拟器测试

- ☒ 启动手机模拟器
- ☒ 启动平板模拟器
- ☒ 测试界面适配效果
- ☒ 观察传感器数据

## 实验结果

### 验证结果

| 功能      | 状态     | 备注                            |
|---------|--------|-------------------------------|
| 天气信息展示  | [x] 成功 | 城市、温度、天气状况、空气质量均正常显示          |
| 多端适配    | [x] 成功 | 手机竖屏(<600px)、平板横屏(≥600px)自动切换 |
| 光线传感器   | [x] 成功 | 实时采集环境光线强度(lux)               |
| 陀螺仪传感器  | [x] 成功 | X/Y/Z三轴数据实时采集                 |
| 分布式数据同步 | [x] 成功 | KV存储跨设备同步正常                   |

## 技术分析报告

### 多端适配分析

- 手机布局特点：垂直排列，紧凑布局
- 平板布局特点：横向排列，更宽敞的空间利用
- 断点系统实现：基于屏幕宽度的响应式设计

### 传感器集成分析

- 光线传感器应用场景：自动亮度调节、环境光感知
- 陀螺仪应用场景：设备姿态检测、游戏控制
- 数据采集准确性：实时性好，精度满足一般应用需求

### 分布式能力分析

- 分布式数据管理原理：基于KV存储的跨设备数据同步
- 跨设备同步实现：通过软总线技术实现设备间通信

# 问题与解决

| 问题描述           | 解决方案                       | 解决结果       |
|----------------|----------------------------|------------|
| 传感器数据无法实时更新到UI | 检查@State装饰器是否正确使用，确保数据绑定生效 | 已解决，数据实时刷新 |
| 平板布局显示异常       | 调整断点阈值，优化Row/Column布局结构    | 已解决，适配正常   |
| 分布式存储初始化失败     | 检查bundleName配置，确保权限配置正确    | 已解决，同步正常   |
| 模拟器传感器无数据      | 使用模拟器提供的传感器模拟功能            | 已解决，数据正常采集 |

# 实验总结

## 实验收获

- 掌握了ArkUI声明式UI开发模式，学会使用@State、@Builder等装饰器构建响应式界面
- 理解了鸿蒙多端适配的断点系统原理，能够根据屏幕宽度实现手机/平板自适应布局
- 掌握了传感器API的调用方法，实现了光线传感器和陀螺仪的数据采集与实时展示
- 了解了分布式数据管理的基本原理，实现了基于KV存储的跨设备数据同步

## 技术要点

- 声明式UI：使用@Entry、@Component、@State等装饰器构建组件，数据驱动视图更新
- 断点适配：通过window.getWindow().getWindowProperties().windowWidth获取屏幕宽度，实现响应式布局
- 传感器调用：使用sensor.on()注册监听，sensor.off()取消监听，实时获取传感器数据
- 分布式存储：使用distributedData.createKVManager创建KV管理器，实现跨设备数据同步

# 考核指标

| 考核项   | 评分标准            | 得分  |
|-------|-----------------|-----|
| 功能完整性 | 天气应用功能完整，多端适配正常 | 25分 |
| 代码质量  | 代码结构清晰，注释完整     | 20分 |
| 多端适配  | 手机/平板界面自适应良好    | 15分 |
| 传感器集成 | 传感器数据采集正常       | 15分 |
| 分布式能力 | 分布式数据同步功能实现     | 15分 |
| 实验报告  | 报告结构完整，内容详实     | 10分 |

# 教师评价

| 项目     | 评价 |
|--------|----|
| 实验完成情况 |    |
| 技术掌握程度 |    |
| 创新点    |    |
| 综合评价   |    |
| 成绩     |    |

教师签名： \_\_\_\_\_

日期： \_\_\_\_\_